IDD IDE

Intent Driven Development

Especificação Técnica — v1.0

IDE baseada no paradigma onde o desenvolvedor declara
intenções, não implementações.

1. Visão Geral

1.1 O Paradigma IDD

Intent Driven Development (IDD) é um paradigma de desenvolvimento onde o desenvolvedor expressa o que o código deve fazer — sua intenção — em vez de descrever como implementá-lo. O IDE captura essa intenção em um formato estruturado, delega a geração de código a um LLM, e monitora continuamente se o código produzido permanece alinhado com a intenção original.

Dimensão	Comparação
Ponto de partida	Código-fonte → Declaração de intenção
Unidade de trabalho	Função / classe → Intent (.intent.yaml)
Rastreabilidade	Commit history → Intent versioning
Testes	Escritos manualmente → Gerados dos critérios
Documentação	Escrita à parte → Derivada da intenção
Revisão de código	Diff de linhas → Drift vs. intenção

1.2 Princípios Fundamentais

- A intenção é a fonte de verdade. Código, testes e documentação são artefatos derivados.
- Todo artefato gerado é rastreável a uma intenção versionada no Intent Store.
- O sistema monitora continuamente o alinhamento entre código e intenção (drift detection).
- Dependências entre intenções são declaradas explicitamente, não inferidas.
- O desenvolvedor decide o quê; o LLM decide o como.

1.3 Base Tecnológica

O IDD IDE é construído sobre o Code - OSS (repositório open source do VS Code), estendido por um conjunto de componentes próprios implementados como extensões VSCode e serviços locais.

Camada	Tecnologia
Base do IDE	Code - OSS (microsoft/vscode)
Linguagem principal	TypeScript
Intent Engine (LLM)	Claude API — claude-sonnet-4-20250514
Formato de intenção	YAML + JSON Schema
Persistência	SQLite — arquivo .idd/store.db
Intent Graph	Cytoscape.js (webview panel)
Integração Git	Hooks: pre-commit, post-merge, post-tag
Testes gerados	Vitest / Jest / Pytest (por linguagem detectada)

2. Arquitetura dos Componentes

O IDD IDE é composto por cinco componentes principais, cada um com responsabilidade única e interface bem definida com os demais.

1 **Intent Capture UI — Painel de Captura de Intenção**
Editor YAML estruturado com autocomplete e validação em tempo real
Painel de constraints: regras de negócio e limites que o código deve respeitar
Critérios de aceite: cada item vira um teste automaticamente via Intent Engine
Grafo de dependências: vinculação explícita a outras intenções (depends_on / used_by)

Preview em tempo real do .intent.yaml gerado antes da submissão ao LLM

2

Intent Engine — Motor de Tradução LLM

Intent Parser: transforma .intent.yaml em prompt rico com papéis semânticos explícitos
Context Manager: injeta contratos das dependências antes de cada chamada ao LLM
LLM Adapter: gerencia chamadas à Claude API com retry, streaming e rate limit
Output Formatter: separa e entrega código, testes, docs e intent_signature
Saída estruturada em JSON validado por schema antes de ser gravada no workspace

3

Code Workspace — Editor Aumentado

Editor Code-OSS com anotação por linha indicando qual intenção gerou o trecho
Diff aumentado: mostra não só o que mudou no código, mas o alinhamento com a intenção
Language Server Protocol (LSP) customizado com diagnósticos de drift inline
Terminal integrado com comandos idd: idd verify, idd regen, idd diff

4

Intent Graph — Grafo Visual de Dependências

Painel lateral com grafo interativo (Cytoscape.js) de todas as intenções do projeto
Nós coloridos por status: alinhado (verde), drift (vermelho), aviso (âmbar), órfã (cinza)
Arestas direcionadas representam depends_on; arestas tracejadas indicam drift propagado
Análise de impacto: ao selecionar um nó, mostra todos os afetados por uma mudança
Exportação como documentação de arquitetura (SVG, PNG, JSON)

5

Intent Verifier + Intent Store — Guardião e Memória

Verifier: processo background conectado ao LSP que analisa drift a cada modificação
Três níveis de alerta: drift crítico (bloqueia commit), aviso (confirmar), ok
Propagação em cascata: drift em uma intenção notifica todas as dependentes
Intent Store: SQLite local com schema normalizado + Git hooks para versionamento
API interna REST acessada pelo Context Manager, Verifier e Intent Graph

3. Formato .intent.yaml

Cada intenção é persistida como um arquivo .intent.yaml no diretório do módulo. Este arquivo é a fonte de verdade que alimenta o Intent Engine e o Intent Store.

3.1 Schema Completo

```
# Campos obrigatórios
intent:       string          # declaração da intenção em linguagem natural
module:       string          # caminho do módulo (ex: auth/login)

# Campos obrigatórios com conteúdo
constraints:  string[]        # restrições e regras de negócio
acceptance:   string[]        # critérios de aceite (viram testes)

# Campos opcionais
depends_on:   string[]        # intenções que este módulo consome
used_by:      string[]        # intenções que dependem deste módulo
language:     string          # linguagem alvo (auto-detectada se omitido)
framework:    string          # framework alvo (ex: express, fastapi)
tags:         string[]        # tags para organização e filtros
version:      semver          # versão atual (gerenciado pelo store)
```

3.2 Exemplo: auth/login

```
intent: "Autenticar usuário com e-mail e senha, retornando JWT válido por 24h"
module: auth/login

constraints:
  - "Senha deve ter mínimo 8 caracteres"
  - "Bloquear conta após 5 tentativas falhas por 15 minutos"
  - "Token JWT deve expirar em exatamente 24h"
  - "Nunca registrar a senha em logs, mesmo em modo debug"

acceptance:
  - "Login válido com credenciais corretas retorna status 200 e token JWT"
  - "Login com senha incorreta retorna 401 sem vazar informações"
  - "Quinta tentativa falha consecutiva bloqueia a conta por 15 min"
  - "Token decodificado contém userId e campo exp válido"

depends_on:
  - users/crud

used_by:
  - dashboard/access
  - auth/refresh-token

language: typescript
framework: express
version: "1.1.0"
```

4. Intent Engine — Fluxo Detalhado

4.1 Estágios do Pipeline

Estágio	Entrada	Saída
1. Intent Parser	.intent.yaml validado	system prompt + user prompt estruturado
2. Context Manager	depends_on list	contratos das dependências em JSON
3. LLM Adapter	prompts + contexto	JSON com code, tests, docs, signature
4. Output Formatter	JSON do LLM	arquivos no workspace + registro no store

4.2 Estrutura do Prompt

O Intent Parser monta dois prompts complementares. O system prompt estabelece o papel do LLM como gerador de código orientado por intenção; o user prompt descreve a intenção com todas as suas dimensões:

```
// System prompt
"Você é um gerador de código para o módulo {module}."
"Gere código que satisfaça a intenção declarada."
"Respeite TODAS as constraints sem exceção."
"Para cada acceptance criterion, gere um teste correspondente."
"Retorne exclusivamente um objeto JSON com os campos:"
"  code, tests, docs, intent_signature"

// User prompt
"Intenção: {intent}"
"Constraints: {constraints[]}"
"Critérios de aceite: {acceptance[]}"
"Stack: {language} + {framework}"
"\nContexto de dependências:\n{deps_context}"
```

4.3 Intent Signature

Ao final de cada geração, o Output Formatter grava uma intent_signature no Intent Store. Esta assinatura é a baseline que o Intent Verifier usa para detectar drift:

```
{
  "intent_hash":     "sha256 do yaml completo",
  "code_hash":       "sha256 do código gerado",
  "generated_at":    "2026-05-17T14:32:00Z",
  "model_used":      "claude-sonnet-4-20250514",
  "criteria_covered": 4,
  "criteria_total":   4,
  "git_commit":      "sha do commit no momento da geração"
}
```

5. Intent Verifier — Detecção de Drift

5.1 Três Análises em Paralelo

A cada modificação no editor (evento onDidChangeTextDocument), o Verifier executa três análises independentes em background:

- Análise semântica por LLM: o trecho modificado e a intent_signature são enviados à API. O modelo retorna um score de alinhamento (0–100%) e lista de constraints violadas. Detecta desvios semânticos que análise estática não alcança.
- Análise estática determinística: verifica presença de funções-chave declaradas nas constraints, ausência de padrões proibidos (ex: log de senhas), e cobertura dos acceptance criteria por testes no arquivo .test.ts correspondente.
- Propagação em cascata: consulta o Intent Graph para todas as intenções em used_by. Se o contrato do módulo atual foi violado, emite alertas secundários nas intenções dependentes.

5.2 Níveis de Alerta

Nível	Condição	Ação do sistema
Drift crítico	Constraint obrigatória violada	Bloqueia commit via pre-commit hook
Aviso	Desvio de constraint não crítica	Alerta inline + ação sugerida
Ok	Alinhamento ≥ 90%	Indicador verde no editor

5.3 Git Hook: pre-commit

```
#!/bin/sh
# .git/hooks/pre-commit — instalado automaticamente pelo IDD IDE

idd verify --staged --fail-on=critical

if [ $? -ne 0 ]; then
  echo ""
  echo "[IDD] Commit bloqueado: drift crítico detectado."
  echo "[IDD] Execute 'idd verify' para ver os detalhes."
  echo ""
  exit 1
fi
```

6. Intent Store — Persistência e API

6.1 Schema SQLite

O Intent Store é um arquivo SQLite em .idd/store.db na raiz do projeto. Quatro tabelas principais:

```
-- Intenções ativas do projeto
CREATE TABLE intents (
  id          TEXT PRIMARY KEY,  -- uuid
  module      TEXT NOT NULL,     -- ex: auth
  sub         TEXT NOT NULL,     -- ex: login
  statement   TEXT NOT NULL,     -- texto da intenção
  status      TEXT NOT NULL,     -- ok | drift | warn | orphan | deprecated
  created_at  TEXT NOT NULL,
  updated_at  TEXT NOT NULL
);

-- Histórico versionado de cada intenção
CREATE TABLE intent_versions (
  id            TEXT PRIMARY KEY,
  intent_id     TEXT REFERENCES intents(id),
  version       TEXT NOT NULL,    -- semver: 1.0.0
  yaml_snapshot TEXT NOT NULL,    -- .intent.yaml completo no momento
  intent_hash   TEXT NOT NULL,    -- sha256 do yaml
  code_hash     TEXT NOT NULL,    -- sha256 do código gerado
  model_used    TEXT NOT NULL,
  git_commit    TEXT,
  created_at    TEXT NOT NULL
);

-- Constraints por intenção
CREATE TABLE constraints (
  id          TEXT PRIMARY KEY,
  intent_id   TEXT REFERENCES intents(id),
  text        TEXT NOT NULL,
  severity    TEXT NOT NULL,  -- critical | warn
  active      INTEGER NOT NULL DEFAULT 1
);

-- Eventos de drift detectados e resolvidos
CREATE TABLE drift_events (
  id             TEXT PRIMARY KEY,
  intent_id      TEXT REFERENCES intents(id),
  constraint_id  TEXT REFERENCES constraints(id),
  type           TEXT NOT NULL,   -- semantic | static | cascade
  detected_at    TEXT NOT NULL,
  resolved_at    TEXT,
  resolution     TEXT            -- fixed | updated_intent | ignored
);
```

6.2 API Interna (HTTP local)

O Intent Store expõe uma API REST local na porta 4999, acessada pelo Context Manager, Verifier e Intent Graph:

Método + Rota	Uso	Retorno
GET /intents	listar todas as intenções	Intent[]
GET /intents/:id/context	Context Manager — enriquece prompt	{ schema, exports, tests }
GET /intents/:id/versions	navegar histórico	IntentVersion[]
POST /intents/:id/versions	gravar nova versão	IntentVersion
POST /drift	registrar evento de drift	DriftEvent
PATCH /drift/:id/resolve	marcar drift como resolvido	DriftEvent
GET /graph	dados para o Intent Graph	{ nodes, edges }
DELETE /intents/:id	deprecar (preserva histórico)	Intent (status=deprecated)

7. Intent Graph — Navegação Visual

O Intent Graph é um painel lateral implementado como VS Code Webview Panel usando a biblioteca Cytoscape.js. Cada intenção é um nó; cada vínculo depends_on / used_by é uma aresta direcionada.

7.1 Informações por Nó

- Identificador: module/sub (ex: auth/login)
- Status visual: cor do nó reflete o estado atual do Verifier
- Badge de drift: marcador vermelho no canto superior direito quando drift ativo
- Painel lateral: ao clicar, exibe constraints, critérios, dependências e ações rápidas

7.2 Codificação Visual de Status

Status	Cor do nó
ok — alinhado com intenção	Verde (#1D9E75)
drift — constraint violada	Vermelho (#E24B4A) + badge + arestas tracejadas
warn — desvio não crítico	Âmbar (#EF9F27)
orphan — sem dependências	Cinza (#888780)
deprecated — fora de uso	Cinza claro com opacidade reduzida

7.3 Propagação Visual de Drift

Quando uma intenção tem drift, todas as arestas que partem dela em direção às dependentes são renderizadas com estilo tracejado e cor vermelha. Isso torna visualmente imediata a pergunta: quais módulos estão em risco por causa deste drift?

8. Roadmap de Implementação

Fase 1 — MVP (semanas 1–8)

- Fork do Code - OSS e configuração do repositório base
- Formato .intent.yaml com JSON Schema e validação no editor
- Extensão: Intent Capture UI (painel de captura com 4 passos)
- Integração básica com Claude API (Intent Engine — estágios 1, 3 e 4)
- Intent Store — schema SQLite e operações CRUD básicas
- Geração de código e testes a partir de uma intenção simples

Fase 2 — Core IDD (semanas 9–18)

- Context Manager — enriquecimento de prompt com dependências
- Intent Verifier — análise estática e detecção básica de drift
- Intent Graph — painel Cytoscape.js com nós, arestas e status
- Git hooks — pre-commit com bloqueio por drift crítico
- Versionamento semântico de intenções no store
- API interna do Intent Store — todos os endpoints

Fase 3 — Produto (semanas 19–28)

- Análise semântica de drift via LLM em background
- Propagação em cascata no Verifier
- Exportação do grafo como documentação
- Suporte a múltiplas linguagens (TypeScript, Python, Go)
- Interface de linha de comando: idd verify, idd regen, idd diff, idd graph
- Marketplace de intent templates para padrões comuns

9. Estrutura do Repositório

```
idd-ide/
├── .idd/                          # diretório de dados do projeto
│   ├── store.db                   # Intent Store (SQLite)
│   └── snapshots/                 # snapshots por release tag
│
├── extensions/
│   └── idd-core/                  # extensão principal do IDE
│       ├── src/
│       │   ├── capture/           # Intent Capture UI (webview)
│       │   ├── engine/            # Intent Engine (parser, adapter, formatter)
│       │   ├── verifier/          # Intent Verifier (LSP + análise)
│       │   ├── graph/             # Intent Graph (Cytoscape webview)
│       │   ├── store/             # Intent Store (SQLite API)
│       │   └── cli/               # comandos idd: verify, regen, diff
│       └── package.json
│
├── src/                           # fonte do Code-OSS (fork)
│
├── product.json                   # branding do IDD IDE
│
└── schemas/
    └── intent.schema.json         # JSON Schema do .intent.yaml
```

10. Dependências e Setup

10.1 Pré-requisitos

- Node.js 20+ (LTS)
- Git 2.40+
- Python 3.11+ (build nativo do Code-OSS)
- Chave de API Anthropic (variável ANTHROPIC_API_KEY)

10.2 Bootstrap

```
# 1. Clonar o repositório
git clone https://github.com/seu-org/idd-ide.git
cd idd-ide

# 2. Instalar dependências
npm install

# 3. Configurar API key
echo 'ANTHROPIC_API_KEY=sk-ant-...' > .env

# 4. Compilar
npm run compile

# 5. Rodar em modo desenvolvimento
./scripts/code.sh

# 6. Instalar Git hooks no projeto alvo
idd init
```

10.3 Primeiro uso em um projeto

```
# Inicializar o IDD no projeto
cd meu-projeto
idd init

# Criar a primeira intenção
idd new auth/login

# Preencher o .intent.yaml gerado e então gerar o código
idd generate auth/login

# Verificar alinhamento a qualquer momento
idd verify

# Ver o grafo no terminal
idd graph --text
```